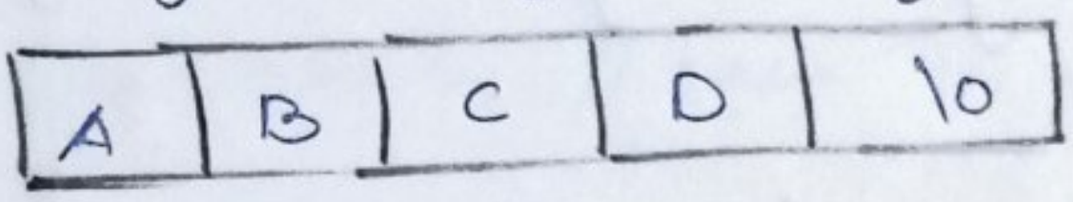
06/06/25

String literal:

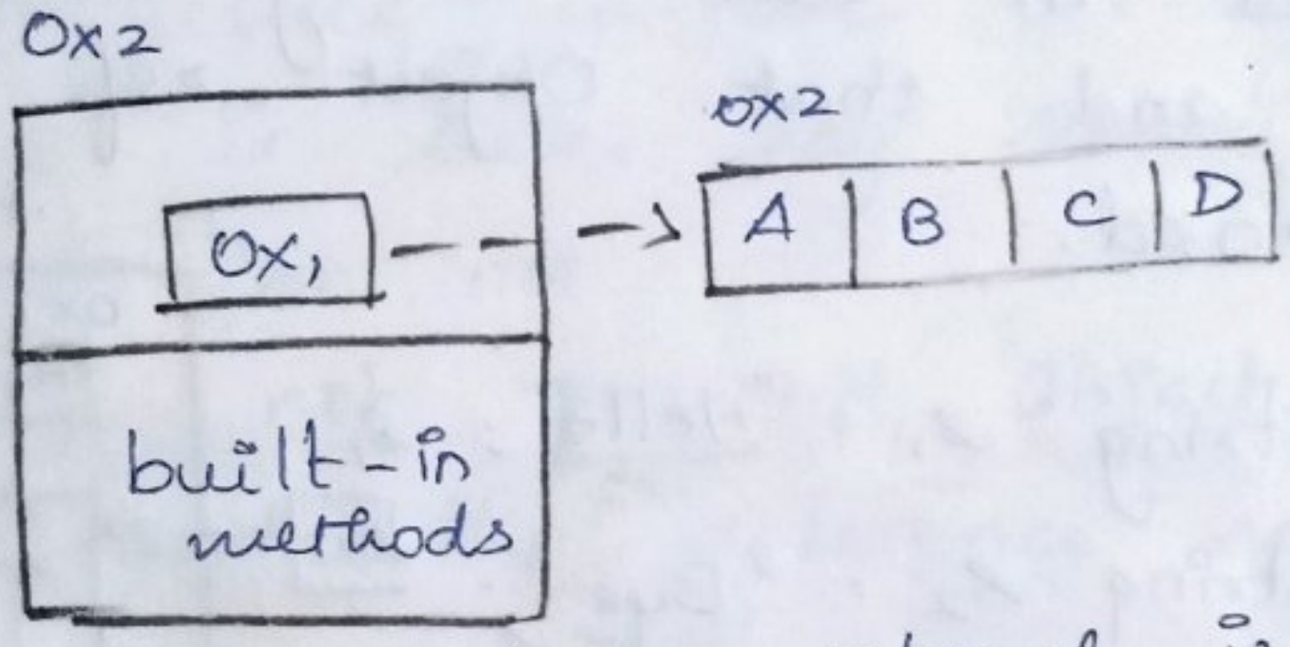
It is a group of characters
(or) multivalue data which will be enclosed with double quotes.

Note:

In c language string literals will be stored by using arrays



In java string literal will be stored by using Objects.



Since the string literal is stored by using Objects, we can have built-in methods to perform the Operations on the string literals.

Note:

Java provides three different blueprints to create Objects to store string literals.

Built-in class:

To create the object to store string type of data

1. String
2. String Buffer
3. String Builder.

Scanned with OKEN Scanner

String Class:

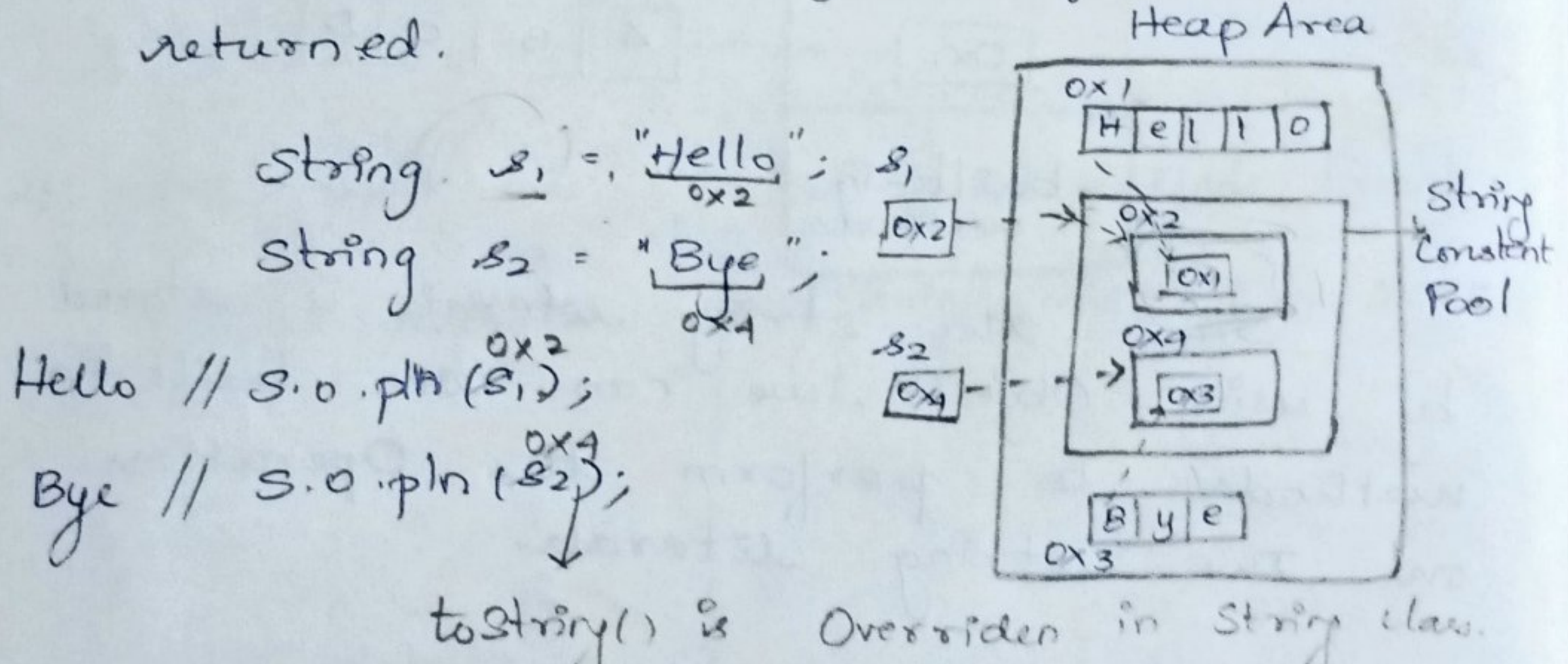
It is a final class which is present inside java.lang package

We can create String Objects in two ways.

1. By using String Literal directly
2. By using new keyword.

1. By using String Literal directly:

Whenever we are using string Literal directly Object will be Created in the String Constant Pool area and that Object ref will be returned.



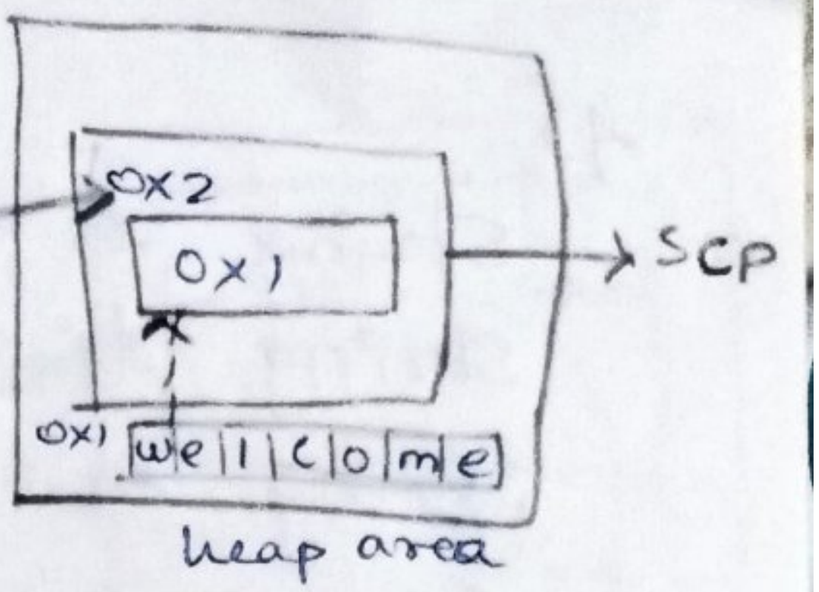
toString() is Overriden in String class.

Note:

If we try to print string ref Variable, we are getting the actual data instead of object reference. Because toString() is Overriden in the String class.

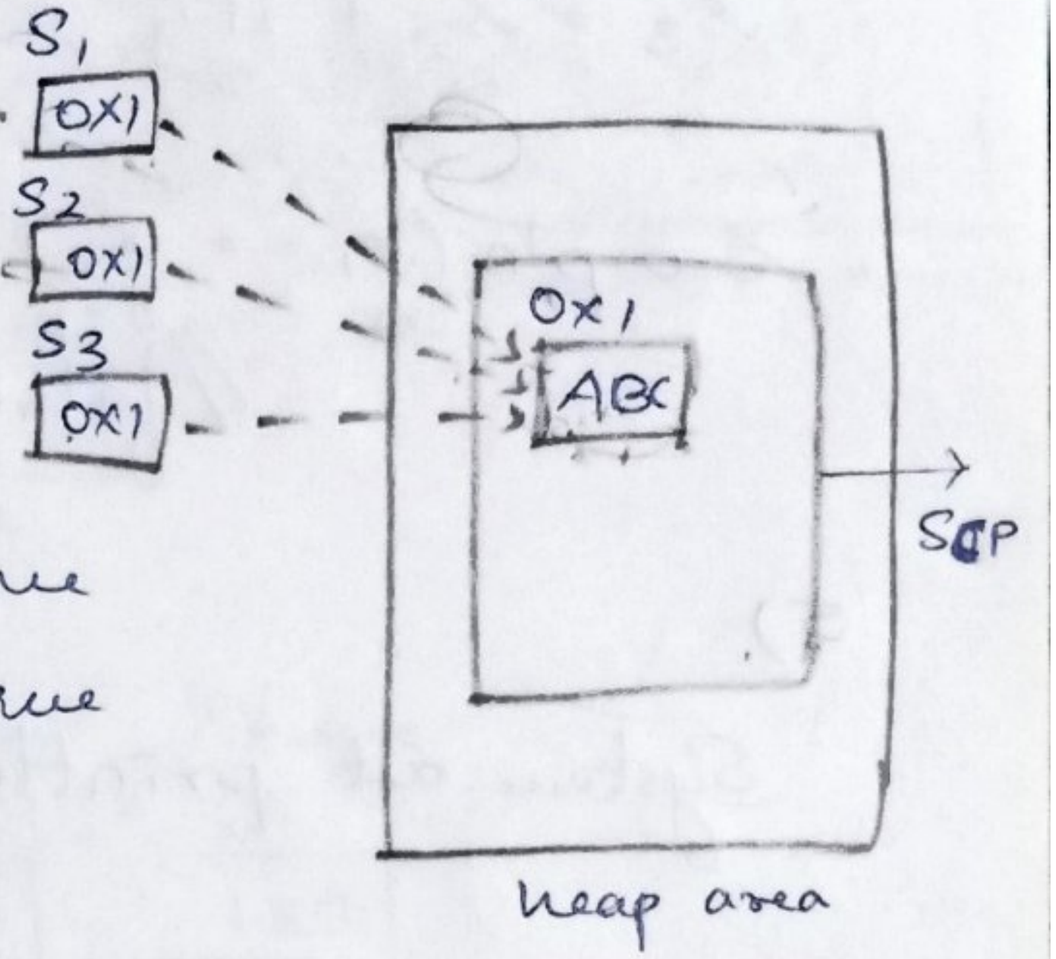
2)

S.o.pln ("Welcome");
0x2



3) String s_1 = "ABC";
String s_2 = s_1;
String s_3 = "ABC";

S.o.pln (s_1 == s_3); // true
S.o.pln (s_2 == s_3); // true



Note:

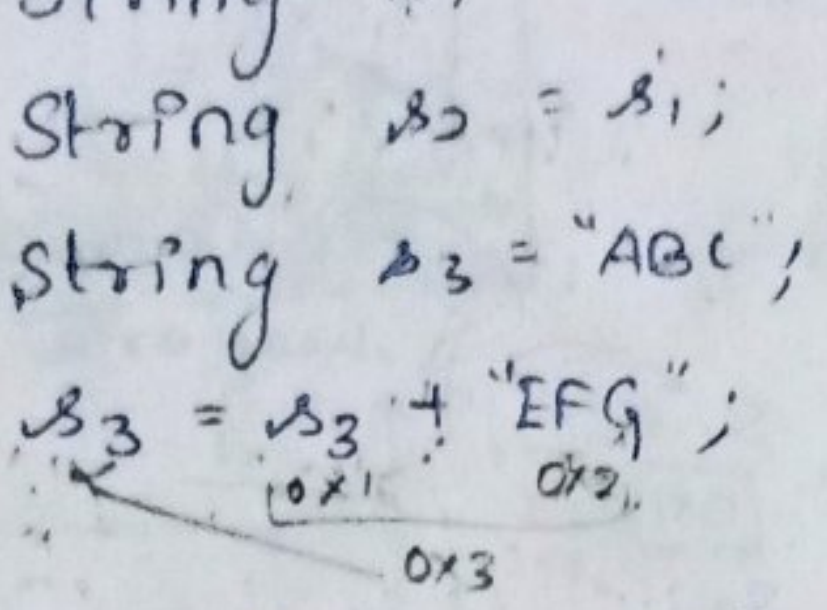
Whenever we are using the String Literal before creating the Object the JVM will check, in the String Constant Pool area is there any same Object is present or not.

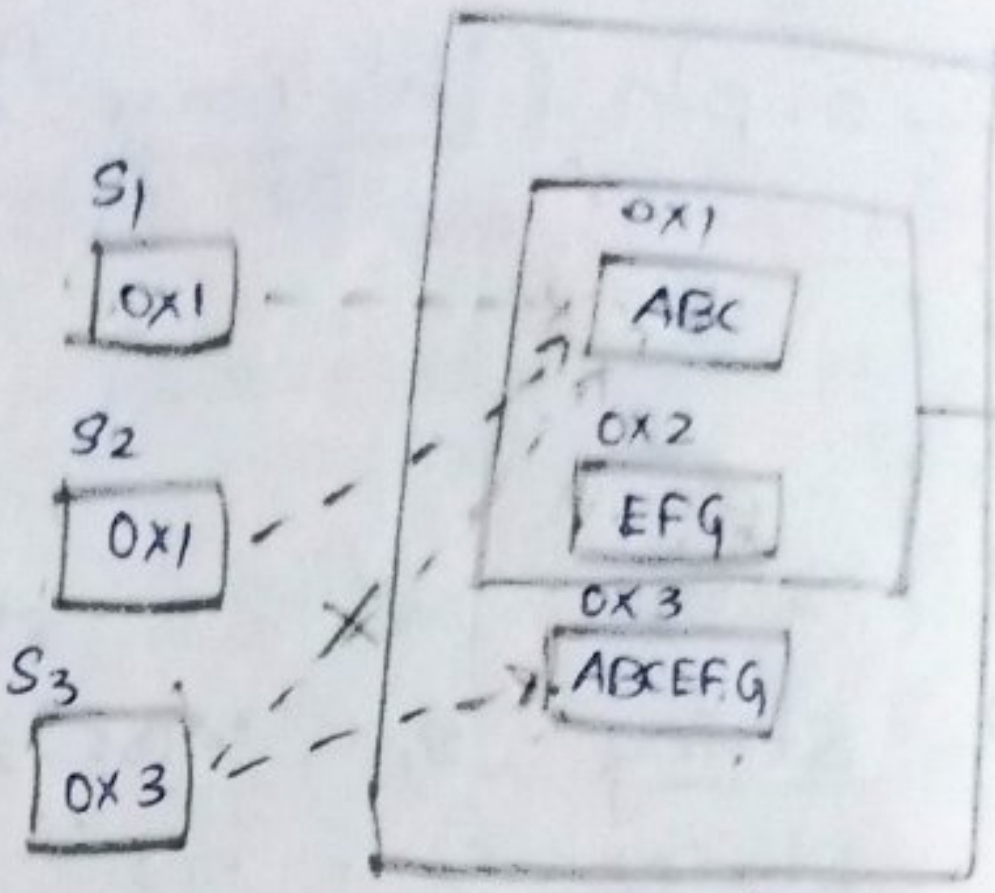
If it is not present, Object will be created and the reference returned else existing Obj reference returned new object will not be Created.

Java has very strong memory management especially with restpect to String type of data.

4)

```
String s1 = "ABC";
String s2 = s1;
String s3 = "ABC";
s3 = s3 + "EFG";
S.o.println (s1 == s3);
```

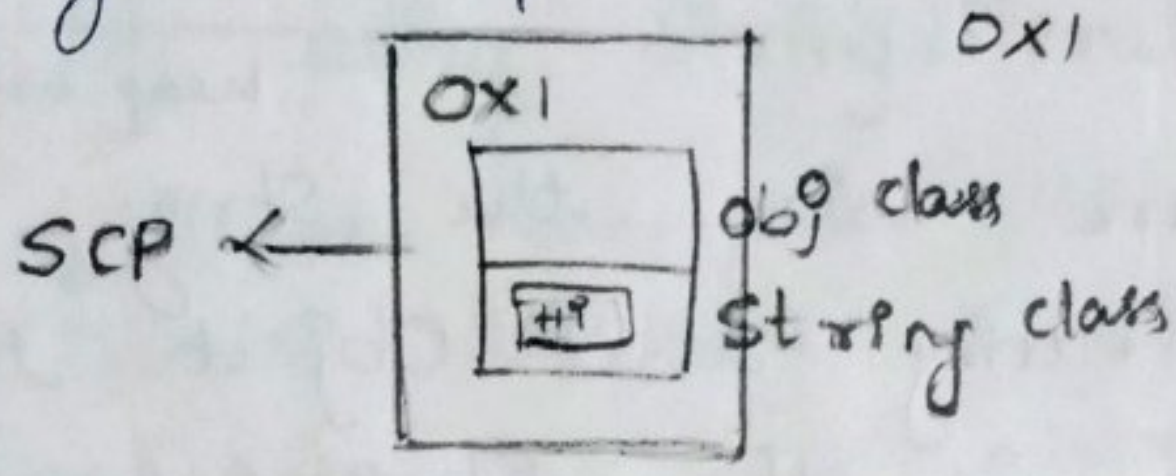




0x1 0x3
// false

5)

System.out.println ("Hi" instanceof Object);
0x1
↓
true.



2) By using new keyword:

In order to use new keyword to create the string Object we need to know the constructors of String class.

Important Constructors of String class:

1. String()
2. String(String str)
3. String (char[] arr)
4. String (byte[] arr)
5. String (StringBuffer sf)
6. String (StringBuilder sb)

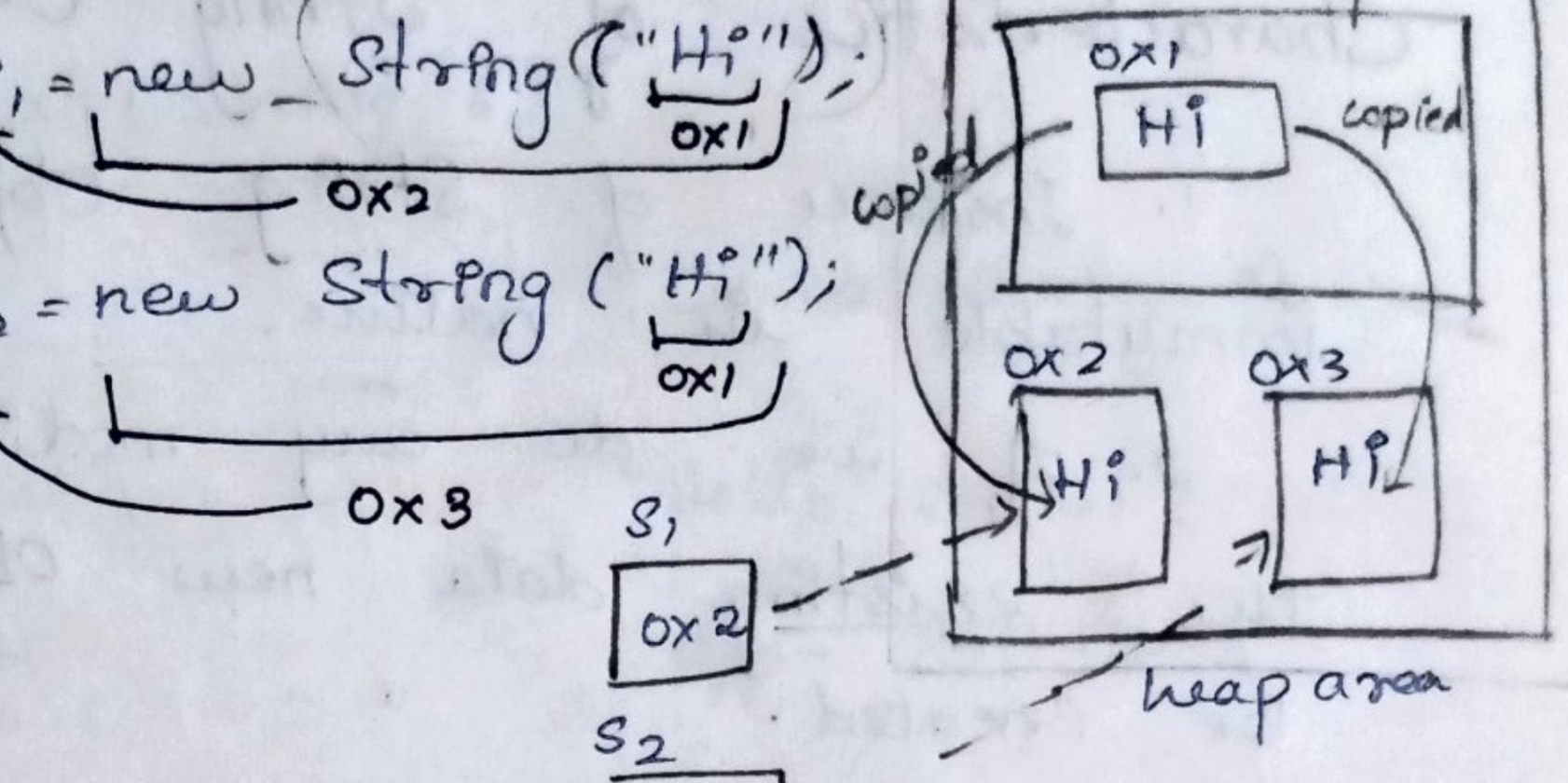
} Convert to String

Scanned with OKEN Scanner

Example:

1) String s1 = new String("Hi");

String s2 = new String("Hi");



S.o.pln(s1 == s2); // false

0x2 == 0x3

S.o.pln(s1.equals(s2)); // true

Hi Hi

Note:

Whenever we are creating string object by using new keyword, the object will be created in the heap area.

equals() is also Overriden in the String class.

hashCode() is also Overriden in String class.

Example:

S.o.pln(s1.hashCode()); // 96354

S.o.pln(s2.hashCode()); // 96354

} same hashcode.

Scanned with OKEN Scanner

Characteristics of String Object:

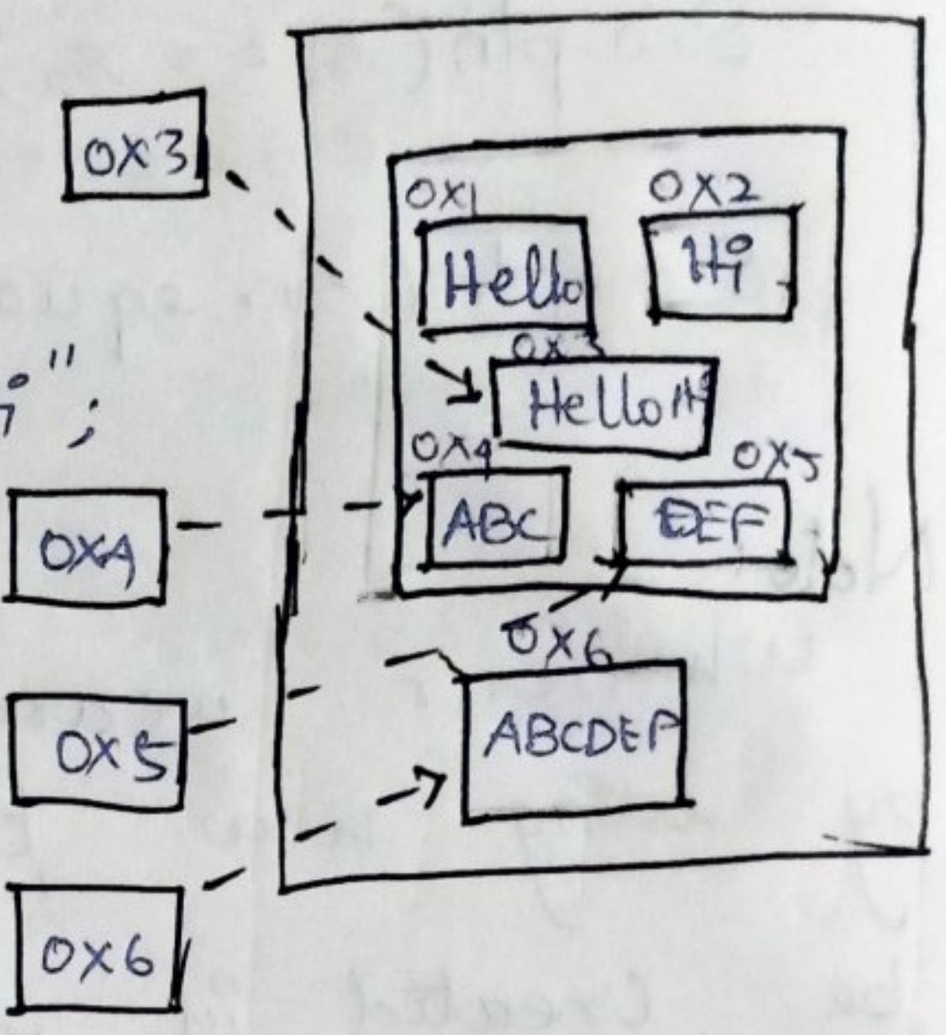
1. Instance of String Object is immutable in nature.
2. If we do any modification on the existing data new Object will be created.

09/06/25

Example:

```
String s1 = "Hello" + "Hi";

String s2 = "ABC";
String s3 = "DEF";
String s4 = s2 + s3;
```



CompareTo(Object o)

returnType is int.

s_1.compareTo(s_2);

if $s_1 == s_2$ return 0 as result.

if $s_1 > s_2$ return +ve as result.

if $s_1 < s_2$ return -ve as result.

Scanned with OKEN Scanner

Difference Between + operator and ~~Compare~~ Concat(String s)

+	Concat(String s)
1) "Hello" + "Hi" o/p- HelloHi	"Hello".concat("Hi"); o/p- HelloHi

2) "Hello" + 'a'
o/p- "Helloa"
"Hello" + true
o/p- "Hellotrue"
	"Hello".concat(10); //CTE
	"Hello".concat('a'); //CTE
	"Hello".concat(true); //CTE

3) "Hello" + null o/p- "Hellonull"	"Hello".concat(null); ↓ nullpointerExceptiom.
4) "Hello" + s, o/p- "HelloFullyQualified@Hexa"	

DrawBack of String Object:

Because of the immutability new Object will be created for each and every modification. So it will consume more memory and the performance of the application will be reduced.

Built-in Methods in String class :

toString()
equals(Object o)
equalsIgnoreCase(Object o)
compareTo(Object o)
hashCode()
toCharArray()
charAt(int index)
getBytes()
trim()
toUpperCase()
toLowerCase()
concat(String s)
substring(int index)
substring(int start, int end)
IndexOf(char ch)
length()
split(String s)
getBytes()
isEmpty()

replaceAll(String s1, String s2)
replace(char ch1, char ch2)
replaceFirst(String s1, String s2)
startsWith(String str)
endsWith(String str)
matches(String str)
contains(CharSequence s)
LastIndexOf(char ch)

Scanned with OKEN Scanner

1) toString()

returntype - String

This method is Overriden in String class. from Object class.

returns the actual data in Strig.

example:

```
String str = "Hello";
System.out.println(str.toString()); // Hello
```

2) equals(Object o)

returntype - Boolean

This method is also Overriden in String from Object class

This method compare the data present in the Object and return true if both are same otherwise return false.

example:

```
String str2 = "Hello";
S.o.pln(str.equals(str2)); //true
```

3) equalsIgnoreCase(Object o)

returntype - Boolean

It compares two strings, ignoring case differences.

example:

```
Strig str3 = "hello";
S.o.pln(st2.equalsIgnoreCase(str3));
                                //true
```

Scanned with OKEN Scanner

4) compareTo (Object o)

returntype : int

It compares two string lexicographically.

returns 0 - if both string are equal

returns +ve number - if string1 is greater than string2.

returns -ve no. - if string1 less than string2.

Exampte :

```
String a = "apple";
String b = "banana";
System.o.pln(a.compareTo(b)); //-1
```

because "apple" < "banana".

```
S.o.pln(a.compareTo(a)); //0
S.o.pln(b.compareTo(a)); //1
```

because "banana" > "apple"

Note :

In Arrays.sort(), compareTo() is called internally.

5) hashCode()

returntype : int

It computes unique integer for the String based on its characters.

This method is also Overriden in String class from Object class.

example :

```
S.o.pln(str.hashCode()); // 12345
S.o.pln(str1.hashCode()); // 12345
```

6) toCharArray()

returntype - characterd Array.

It is used to convert a string into a char array.

example.

```
char[] arr = str.toCharArry();
```

↓

[H, E, l, l, o]

7) CharAt(int index)

returntype - char

It is used to get the char at a specific index in a string.

example:

```
S.o.pln (str.charAt(1)); // e
```

8) getBytes()

returntype - byte[] array

It is used to convert string into sequence of bytes, using specific character ASCII value.

example:

```
byte[] b = str.getBytes()
```

↓

[72, 101, 108, 108, 111]

9) trim()

returntype - string

It is used to remove leading and trailing whitespace from a string.

example:

```
string e = "  Hello  ";
s.o.pln(e.trim()); // "Hello"
```

10) toUpperCase()

returntype - string

It is used to convert all char in a string to uppercase.

example:

```
string u = e.toUpperCase();
System.out.println(u); // HELLO
```

11) toLowerCase()

returntype - string

It is used to convert all char in a string to lowercase.

example:

```
string l = u.tolowerCase();
S.o.pln(l); // hello
```

12) concat(string s)

returntype - string

It is used to join one string to the end of another.

example:

```
S.o.pln(u.concat(l)); // HELLOhello
```

Scanned with OKEN Scanner

13) substring (int index)

returntype - string.

It is used to extract a part of the string, starting from the given index to the end of the string.

example:

S.o.pln (str.substring (1)); // ello

14) substring (int start, int end)

returntype - string.

It is used to extract a portion of string between given start & end-1 indices.

example:

S.o.pln (str.substring (1,4)); // ell

15) indexOf (char ch)

returntype - int

It is used to find the index of the first occurence of a given char in a string.

example:

S.o.pln (str.index ('l')); // 2

16) lastIndexOf (char ch)

returntype - int

find the index of last Occurrence of given char in a string.

example:

S.o.pln (str.lastIndexOf ('l')); // 3

Scanned with OKEN Scanner

17) length()

returntype - int

It is used to get the number of character in String.

example:

S.o.pln(str.length()); // 5

18) split(string s)

returntype - String[] array

It is used to split a string into an array of substrings based on given separator.

example:

String st = "Java is fun";

String[] words = st.split(" ");

↓

["Java", "is", "fun"]

String st2 = "Java---is--Good";

String word2 = st2.split("[]+");

↓

["Java", "is", "Good."]

Note:

In split() we can use Regular Expression.

19) isEmpty()

returntype = boolean

used to check whether string is empty or not.

example:

S.o.pln("".isEmpty()); // true

S.o.pln(str.isEmpty()); // false

20) replaceAll (String s1, String s2)

returntype - String

It is used to replace all the parts of string that match a pattern (s1) with another String (s_2)

example:

String s_1 = "abbacbbd";

S.o.pln (s_1. replaceAll ("bb", "ee");

// aeeaceed

21) replace (char ch1, char ch2)

returntype : String

used to replace all the Occurences of one char (c_1) in a string with another char (c_2).

example:

S.o.pln (s1. replace ('a', 'f'));

// fbbfcbbd

22) replaceFirst (String s1, String s2)

returntype - String

It is used to replace first Occurence or only the first match of the pattern s1 in string with s2.

example:

S.o.pln (s1. replaceFirst ("bb", "ee"));

// aeeacbbd

Scanned with OKEN Scanner

23) startsWith (String str)

returntype - boolean

It check if a string begins with the specified prefix str.

example:

"ABC". startsWith ("AB"); // true.

24) endsWith (String str)

returntype - boolean

It check if a string ends with the specified suffex str.

example:

"ABc". endsWith ("Bc"); //true

25) matches (String str)

returntype - boolean

It checks if the entire string exactly matches the given regular expression str.

example:

"Hi". matches ("Hi"); // true

"H", matches ("[A-Z]"); //true

"H1". matches ("[A-Za-z0-9]"); //false

"H1". matches ("[A-Za-z0-9]+"); //true

"012". matches ("[^a-z]+")); //true

26) Contains (CharSequence s)

returntype - boolean

used to check of the String

Include the given sequence of Char

s.

example:

```
String c = "Java programming";
S.o.pln ( c.contains ("gram")); // true
S.o.pln ( c. contains ("hello")); // false
S.o.pln (c. Contains ( "[a-zA-Z]+ ")); // true
```

CharSequence:

It is an Parent Interface for String, StringBuffer, StringBuilder.

It have the common methods which is used to string, isb, sbuilder. like length(), isEmpty(), etc,...

Regular Expression:

It is a pattern used to match or search for specific sequence of character within string

we can use regular Expression in split(), replaceAll(), matches().

Scanned with OKEN Scanner

Note:

To Overcome this drawBack we need to create mutable Objects.

In java we can create mutable Objects to string literal by using StringBuffer and StringBuilder.

StringBuffer :

It is a final class which is present inside java.lang package.

How to Create StringBuffer Objects:

By using new keyword. only.

Constructors in StringBuffer:

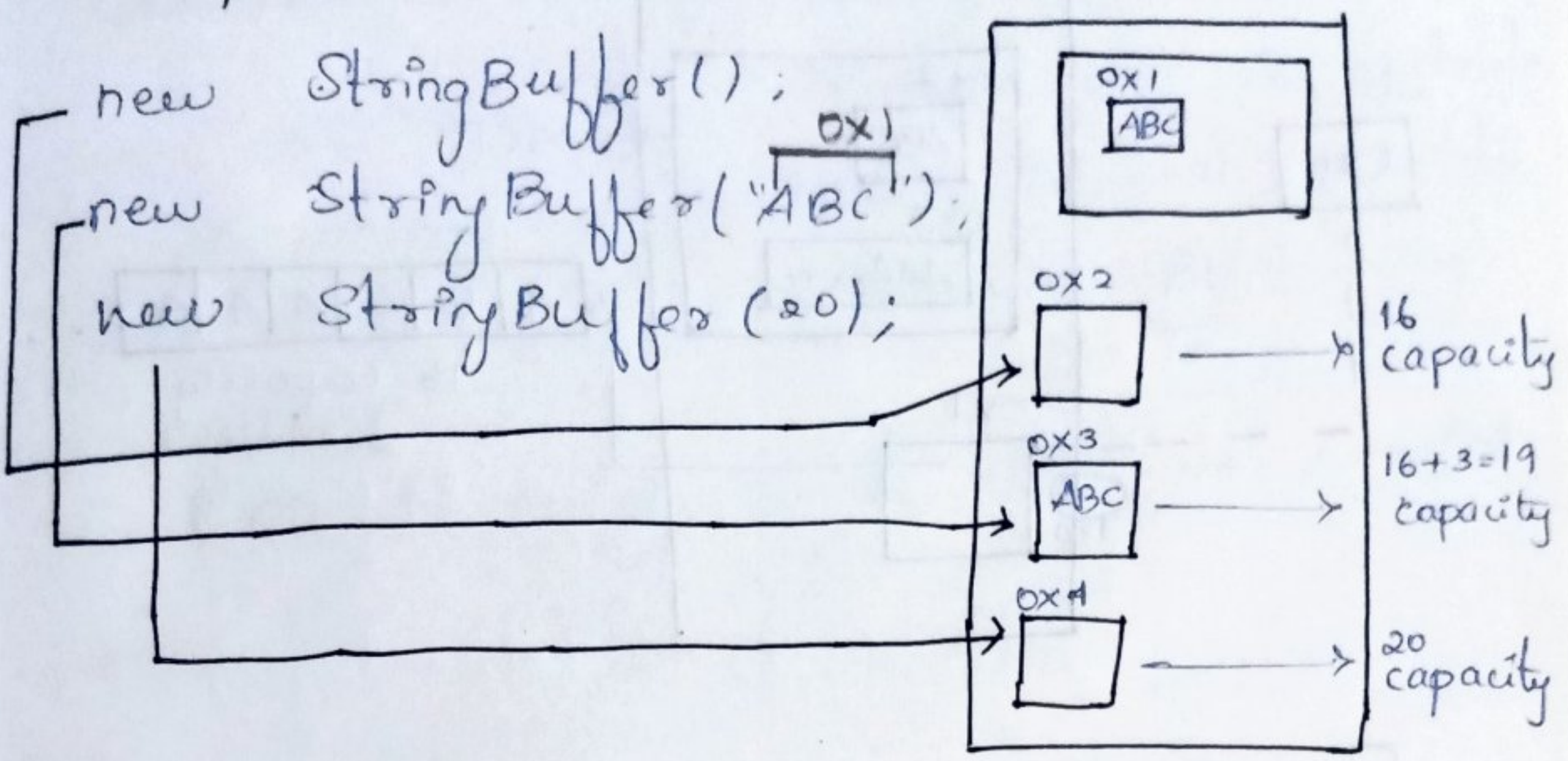
1. StringBuffer()
2. StringBuffer(String str)
3. StringBuffer(int intial capacity)

Note :

The default capacity of an empty StringBuffer Object is 16.

Scanned with OKEN Scanner

Example:



Example:

StringBuffer sb = new StringBuffer();

S.o.pln(sb.capacity()); - 16

S.o.pln(sb.length()); - 0

~~S.o.pln~~ sb.append("ABC");

S.o.pln(sb.capacity()); - 16

S.o.pln(sb.length()); - 3

sb.append("AAAAAAAAAAAAAAA");

S.o.pln(sb.capacity()); - 16

S.o.pln(sb.length()); - 16

sb.append("a");

S.o.pln(sb.capacity()); - 34

S.o.pln(sb.length()); - 17

sb.append("yy....") (17);

S.o.pln(sb.capacity()); - 34

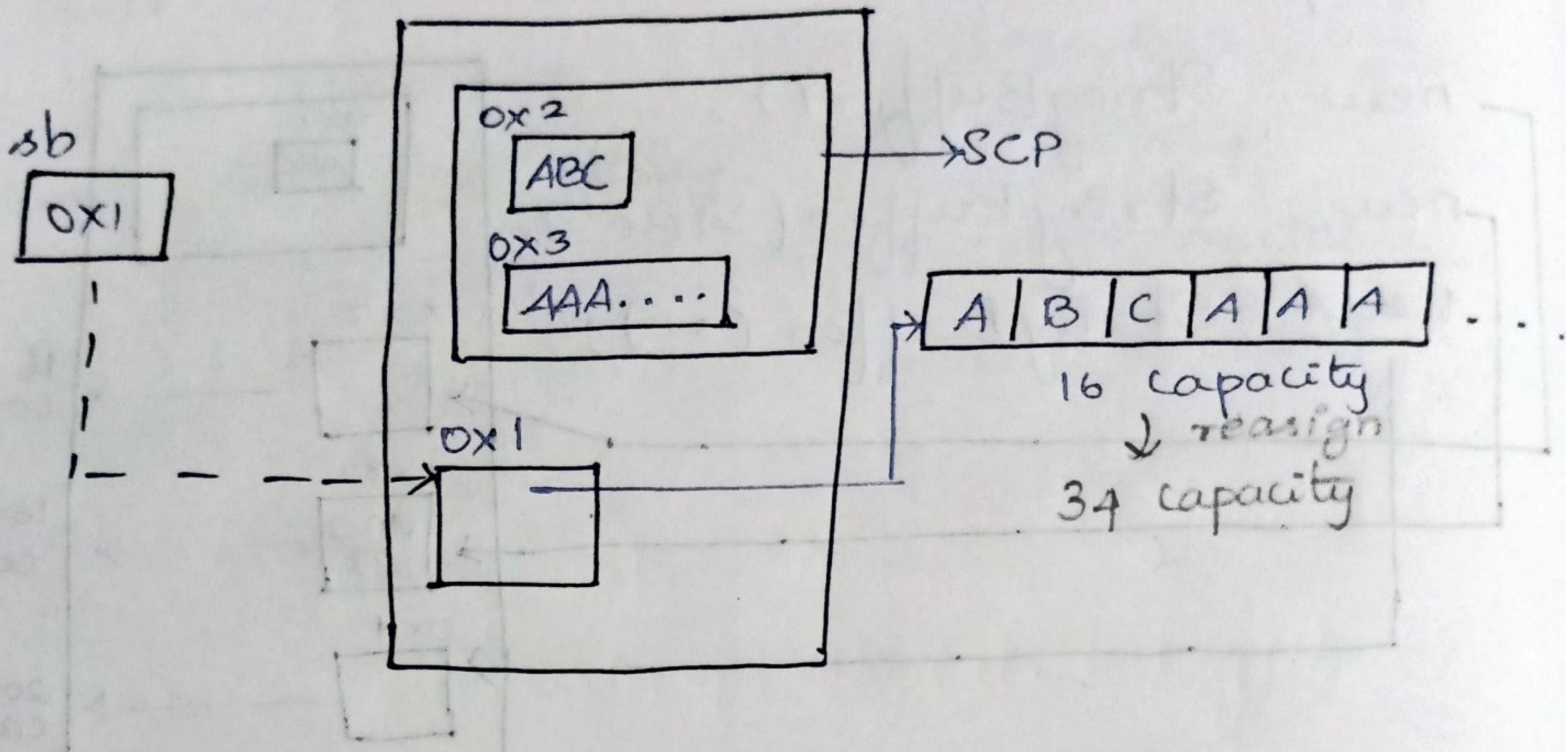
S.o.pln(sb.length()); - 34

sb.append("z");

S.o.pln(sb.capacity()); - 35

S.o.pln(sb.length()); - 35

Scanned with OKEN Scanner



Formula

new capacity = (old capacity + 1) * 2

= (16 + 1) * 2

= 34

Example:

StringBuffer sb1 = new StringBuffer("ABC");

StringBuffer sb2 = new StringBuffer("ABC");

StringBuffer sb3 = sb1 + sb2 // CTE

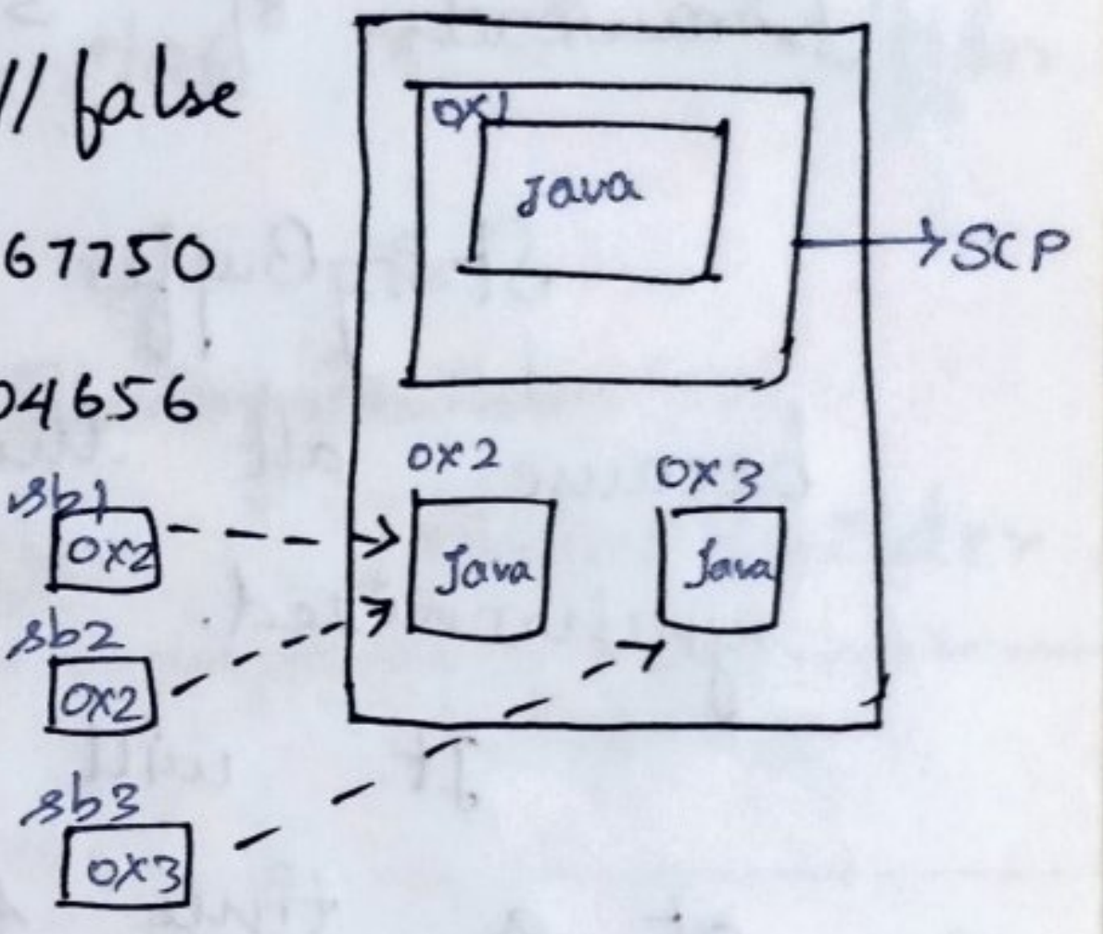
Characteristics of StringBuffer:

1. Instance of StringBuffer is mutable.
2. String Concatenation is not possible with respect to StringBuffer By using + operator.
3. StringBuffer Object will be Created in the heap area.

10/06/25.

Example:

```
StringBuffer sb1 = new StringBuffer("java");
StringBuffer sb2 = sb1;
S.o.pln(sb1 == sb2); // true
StringBuffer sb3 = new StringBuffer("java");
S.o.pln(sb1 == sb3); // false
S.o.pln(sb1.equals(sb3)); // false
S.o.pln(sb1.hashCode()); 1867750
S.o.pln(sb3.hashCode()); 204656
S.o.pln(sb1); // Java
S.o.pln(sb2); // Java
```



Note:

→ toString() is Overriden in the StringBuffer.

Note:

→ hashCode() & equals() are not Overriden in StringBuffer.

Note:

We need to convert StringBuffer into String to compare the Data.

example:

new String(sb1).equals(new String(sb3))

(or)

sb1.toString().equals(sb3.toString())

Scanned with OKEN Scanner

To generate unique Integer Based on states

example:

sb1.toString().hashCode()
sb3.toString().hashCode()
} same hashcode.

DrawBacks of StringBuffer:

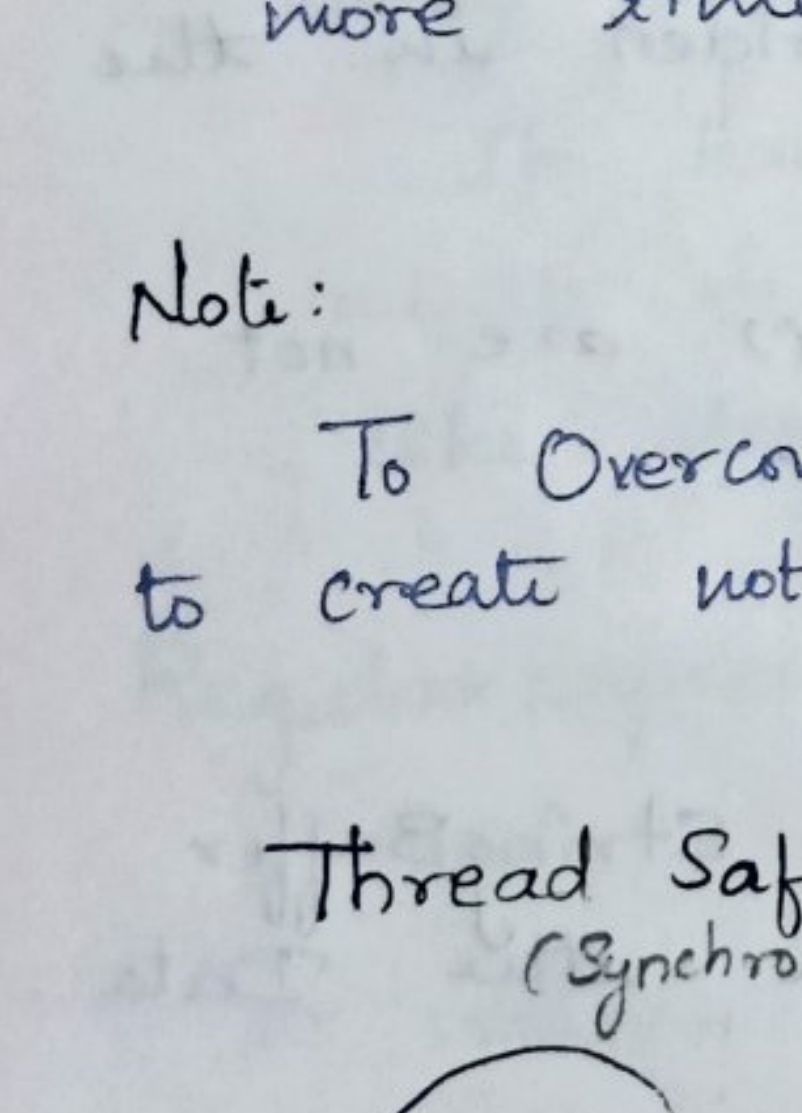
StringBuffer Objects are thread safe because all the methods are synchronized.

It will allow only one task at a time so that it will take more time to execute multiple task.

Note:

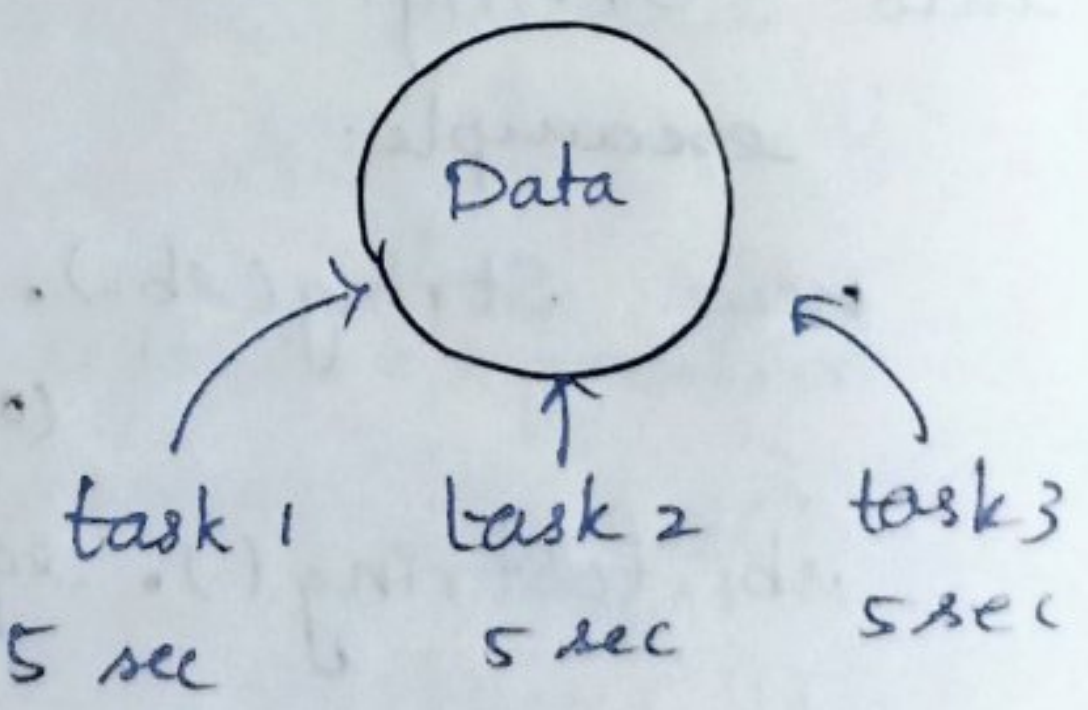
To Overcome this problem we need to create not thread safe Object (StringBuilder).

Thread Safe
(Synchronized)



Total = 15 sec

Not Thread Safe



Total - 5 sec

i

StringBuilder:

It is a final class which is present inside java.lang package.

Characteristics:

→ StringBuilder objects are not Thred safe. Except this all the remaining characteristics are similar to StringBuffer.

~~StringBuffer~~ String	StringBuffer	~~StringBuilder~~ StringBuilder
immutable	mutable	mutable
we can String obj by using literals directly aswell as by using new keyword.	only By using new keyword.	only By using new keyword.
There is no default capacity. toString(), equals() & hashCode() are Overriden.	The default capacity is 16. only toString() is Overriden.	The default capacity is 16. only toString() is Overriden.
String concatination is possible by using + operator.	string concatination is not possible by using + Operator.	Strig Concatination is not possible by using + Operator.
The Obj will be created in both SCP & Heap area.	The SB Obj will be created only inside heap area	The obj will be created only inside heap area.

Low Performance	Medium Performance	High Performance
All methods are not synchronized.	All methods are synchronized.	All methods are not synchronized.
String obj are Thread safe. (Because of the immutab-lity)	StringBuffer obj are thread safe because all methods are synchronized.	Stringbuilder obj are not thread safe because of all methods are not synchronized
when there is no frequent modification on the String Literal, we can go with String.	when there is a need of frequent modification, we can go with StringBuffer.	when there is need of frequent modification & multithreading we can go with StringBuilder.

How to Create immutable Class/Obj:

Create the class as final.

Make all the attributes as private final.

Initialize the attribute by using constructor.

Provide only getter(), don't provide setter().

Scanned with OKEN Scanner